

SCUT Data Analysis And Modeling - Lab 2

Introduction to data

Use the template

Before you begin the lab, don't forget to use the template used in the previous lab lesson.

Getting started

Some define Statistics as the field that focuses on turning information into knowledge. The first step in that process is to summarize and describe the raw information - the data. In this lab, you will gain insight into public health by generating simple graphical and numerical summaries of a data set collected by the Centers for Disease Control and Prevention (CDC). As this is a large data set, along the way you'll also learn the indispensable skills of data processing and subsetting.

The Behavioral Risk Factor Surveillance System (BRFSS) is an annual telephone survey of 350,000 people in the United States. As its name implies, the BRFSS is designed to identify risk factors in the adult population and report emerging health trends. For example, respondents are asked about their diet and weekly physical activity, their HIV/AIDS status, possible tobacco use, and even their level of healthcare coverage. The BRFSS Web site (<http://www.cdc.gov/brfss>) contains a complete description of the survey, including the research questions that motivate the study and many interesting results derived from the data.

We will focus on a random sample of 20,000 people from the BRFSS survey conducted in 2000. While there are over 200 variables in this data set, we will work with a small subset.

We begin by loading the data set of 20,000 observations into the R workspace. After launching RStudio, enter the following command.

```
load(url("http://www.stat.duke.edu/~cr173/Sta102_Sp16/Lab/cdc.Rdata"))
```

The data set `cdc` that shows up in your workspace is a *data matrix*, with each row representing a *case* and each column representing a *variable*. R calls this data format a *data frame*, which is a term that will be used throughout the labs.

To view the names of the variables, type the command

```
names(cdc)
```

This returns the names `genhlth`, `exerany`, `hlthplan`, `smoke100`, `height`, `weight`, `wtdesire`, `age`, and `gender`. Each one of these variables corresponds to a question that was asked in the survey. A list of the variables and what they represent is given below:

- `genhlth` : general health - excellent, very good, good, fair or poor
- `exerany` : whether the respondent exercised in the past month (1) or did not (0)
- `hlthplan` : whether the respondent had some form of health coverage (1) or did not (0)
- `smoke100` : whether the respondent had smoked at least 100 cigarettes in her lifetime
- `height` : height, in inches
- `weight` : weight, in pounds

- `wt_desire` : desired weight, in pounds
- `age` : age, in years
- `gender` : gender, male (m) or female (f)

Exercise 1 How many cases are there in this data set? How many variables? You can use the 'dim' code to answer and use text to explain. For each variable, identify its data type (e.g. categorical, discrete).

We can have a look at the first few entries (rows) of our data with the command

```
head(cdc)
```

and similarly we can look at the last few by typing

```
tail(cdc)
```

You could also look at *all* of the data frame at once by typing its name into the console, but that might be unwise here. We know `cdc` has 20,000 rows, so viewing the entire data set would mean flooding your screen. It's better to take small peeks at the data with `head`, `tail` or the subsetting techniques that you'll learn in a moment.

Summaries and tables

The BRFSS questionnaire is a massive trove of information. A good first step in any analysis is to distill all of that information into a few summary statistics and graphics. As a simple example, the function `summary` returns a numerical summary: minimum, first quartile, median, mean, second quartile, and maximum. For `weight` this is

```
summary(cdc$weight)
```

You can view some information through "Help".

```
?summary
```

R also functions like a very fancy calculator. If you wanted to compute the interquartile range for the respondents' weight, you would look at the output from the summary command above and then enter

```
190 - 140
```

R also has built-in functions to compute summary statistics one by one. For instance, to calculate the mean, median, variance and standard deviation of `weight`, type

```
mean(cdc$weight)
var(cdc$weight)
sd(cdc$weight)
median(cdc$weight)
```

While it makes sense to describe a quantitative variable like `weight` in terms of these statistics, what about categorical data? We would instead consider the sample frequency or relative frequency distribution. The function `table` does this for you by counting the number of times each kind of response was given. For example, to see the number of people who have smoked 100 cigarettes in their lifetime, type

```
table(cdc$smoke100)
```

or instead look at the relative frequency distribution by typing

```
table(cdc$smoke100)/20000
```

Notice how R automatically divides all entries in the table by 20,000 in the command above. This is similar to something we observed in the last lab; when we multiplied or divided a vector with a number, R applied that action across entries in the vectors. As we see above, this also works for tables. Next, we make a bar plot of the entries in the table by putting the table inside the `barplot` command.

```
barplot(table(cdc$smoke100))
```

Notice what we've done here! We've computed the table of `cdc$smoke100` and then immediately applied the graphical function, `barplot`. This is an important idea: R commands can be nested. You could also break this into two steps by typing the following:

```
smoke <- table(cdc$smoke100)
```

```
barplot(smoke)
```

Here, we've made a new object, a table, called `smoke` (the contents of which we can see by typing `smoke` into the console) and then used it in as the input for `barplot`. The special symbol `<-` performs an *assignment*, taking the output of one line of code and saving it into an object in your workspace. This is another important idea that we'll return to later.

Exercise 2 Create a numerical summary for `height` and `age`, and compute the interquartile range for each. Compute the relative frequency distribution for `gender` and `exerany`. How many males are in the sample? What proportion of the sample reports being in excellent health?

The `table` command can be used to tabulate any number of variables that you provide. For example, to examine which participants have smoked across each gender, we could use the following.

```
table(cdc$gender, cdc$smoke100)
```

Here, we see column labels of 0 and 1. Recall that 1 indicates a respondent has smoked at least 100 cigarettes. The rows refer to gender. To create a mosaic plot of this table, we would enter the following command.

```
mosaicplot(table(cdc$gender, cdc$smoke100))
```

We could have accomplished this in two steps by saving the table in one line and applying `mosaicplot` in the next (see the table/barplot example above).

Exercise 3 What does the mosaic plot reveal about smoking habits and gender?

Interlude: How R thinks about data

We mentioned that R stores data in data frames, which you might think of as a type of spreadsheet. Each row is a different observation (a different respondent) and each column is a different variable (the first is `genhlth`, the second `exerany` and so on). We can see the size of the data frame next to the object name in the workspace or we can type

```
dim(cdc)
```

which will return the number of rows and columns. Now, if we want to access a subset of the full data frame, we can use row-and-column notation. For example, to see the sixth variable of the 567th respondent, use the format

```
cdc[567, 6]
```

which means we want the element of our data set that is in the 567th row (meaning the 567th person or observation) and the 6th column (in this case, weight). We know that `weight` is the 6th variable because it is the 6th entry in the list of variable names

```
names(cdc)
```

To see the weights for the first 10 respondents we can type

```
cdc[1:10, 6]
```

In this expression, we have asked just for rows in the range 1 through 10. R uses the `:` to create a range of values, so `1:10` expands to 1, 2, 3, 4, 5, 6, 7, 8, 9, 10. You can see this by entering

```
1:10
```

Finally, if we want all of the data for the first 10 respondents, type

```
cdc[1:10, ]
```

By leaving out an index or a range (we didn't type anything between the comma and the square bracket), we get all the columns. When starting out in R, this is a bit counterintuitive. As a rule, we omit the column number to see all columns in a data frame. Similarly, if we leave out an index or range for the rows, we would access all the observations, not just the 567th, or rows 1 through 10. Try the following to see the weights for all 20,000 respondents fly by on your screen

```
cdc[, 6]
```

Recall that column 6 represents respondents' weight, so the command above reported all of the weights in the data set. An alternative method to access the weight data is by referring to the name. Previously, we typed `names(cdc)` to see all the variables contained in the `cdc` data set. We can use any of the variable names to select items in our data set.

```
cdc$weight
```

The dollar-sign tells R to look in data frame `cdc` for the column called `weight`. Since that's a single vector, we can subset it with just a single index inside square brackets. We see the weight for the 567th respondent by typing

```
cdc$weight[567]
```

Similarly, for just the first 10 respondents

```
cdc$weight[1:10]
```

The command above returns the same result as the `cdc[1:10, 6]` command. Both row-and-column notation and dollar-sign notation are widely used, which one you choose to use depends on your personal preference.

A little more on subsetting

It's often useful to extract all individuals (cases) in a data set that have specific characteristics. We accomplish this through *conditioning* commands. First, consider expressions like

```
cdc$gender == "m"
```

or

```
cdc$age > 30
```

These commands produce a series of `TRUE` and `FALSE` values. There is one value for each respondent, where `TRUE` indicates that the person was male (via the first command) or older than 30 (second command).

Suppose we want to extract just the data for the men in the sample, or just for those over 30. We can use the R function `subset` to do that for us. For example, the command

```
mdata <- subset(cdc, cdc$gender == "m")
```

will create a new data set called `mdata` that contains only the men from the `cdc` data set. In addition to finding it in your workspace alongside its dimensions, you can take a peek at the first several rows as usual

```
head(mdata)
```

This new data set contains all the same variables but just under half the rows. It is also possible to tell R to keep only specific variables, which is a topic we'll discuss in a future lab. For now, the important thing is that we can carve up the data based on values of one or more variables.

As an aside, you can use several of these conditions together with `&` and `|`. The `&` is read “and” so that

```
m_and_over30 <- subset(cdc, gender == "m" & age > 30)
```

will give you the data for men over the age of 30. The `|` character is read “or” so that

```
m_or_over30 <- subset(cdc, gender == "m" | age > 30)
```

will take people who are men or over the age of 30 (why that's an interesting group is hard to say, but right now the mechanics of this are the important thing). In principle, you may use as many “and” and “or” clauses as you like when forming a subset.

Exercise 4

Create a new object called `under23_and_smoke` that contains all observations of respondents under the age of 23 that have smoked 100 cigarettes in their

lifetime. Write the command you used to create the new object as the answer to this exercise.

Quantitative data

With our subsetting tools in hand, we'll now return to the task of the day: making basic summaries of the BRFSS questionnaire. We've already looked at categorical data such as `smoke` and `gender` so now let's turn our attention to quantitative data. Two common ways to visualize quantitative data are with box plots and histograms. We can construct a box plot for a single variable with the following command.

```
boxplot(cdc$height)
```

You can compare the locations of the components of the box by examining the summary statistics.

```
summary(cdc$height)
```

Confirm that the median and upper and lower quartiles reported in the numerical summary match those in the graph. The purpose of a boxplot is to provide a thumbnail sketch of a variable for the purpose of comparing across several categories. So we can, for example, compare the heights of men and women with

```
boxplot(cdc$height ~ cdc$gender)
```

The notation here is new. The `~` character can be read *versus* or *as a function of*. So we're asking R to give us a box plots of heights where the groups are defined by gender.

Next let's consider a new variable that doesn't show up directly in this data set: Body Mass Index (BMI) (http://en.wikipedia.org/wiki/Body_mass_index (http://en.wikipedia.org/wiki/Body_mass_index)). BMI is a weight to height ratio and can be calculated as:

$$BMI = \frac{weight\ (lb)}{height\ (in)^2} * 703$$
$$BMI = \frac{weight\ (lb)}{height\ (in)^2} * 703$$

703 is the approximate conversion factor to change units from metric (meters and kilograms) to imperial (inches and pounds).

The following two lines first make a new object called `bmi` and then creates box plots of these values, defining groups by the variable `cdc$genhlth`.

```
bmi <- (cdc$weight / cdc$height^2) * 703  
boxplot(bmi ~ cdc$genhlth)
```

Notice that the first line above is just some arithmetic, but it's applied to all 20,000 numbers in the `cdc` data set. That is, for each of the 20,000 participants, we take their weight, divide by their height-squared and then multiply by 703. The result is 20,000 BMI values, one for each respondent. This is one reason why we like R: it lets us perform computations like this using very simple expressions.

Exercise 5 What does this box plot show? Pick another categorical variable from the data set and see how it relates to BMI. List the variable you chose, why you might

think it would have a relationship to BMI, and indicate what the figure seems to suggest.

Finally, let's make some histograms. We can look at the histogram for the age of our respondents with the command

```
hist(cdc$age)
```

Histograms are generally a very good way to see the shape of a single distribution, but that shape can change depending on how the data is split between the different bins. You can control the number of bins by adding an argument to the command. In the next two lines, we first make a default histogram of `bmi` and then one with 50 breaks.

```
hist(bmi)
hist(bmi, breaks = 50)
```

Note that you can flip between plots that you've created by clicking the forward and backward arrows in the lower right region of RStudio, just above the plots. How do these two histograms compare?

At this point, we've done a good first pass at analyzing the information in the BRFSS questionnaire. We've found an interesting association between smoking and gender, and we can say something about the relationship between people's assessment of their general health and their own BMI. We've also picked up essential computing tools – summary statistics, subsetting, and plots – that will serve us well throughout this course.

Data manipulation

Sometimes you might need to alter the dataset you have in some way to make it better serve your analysis goals. In this section you will learn a few useful “data wrangling” skills. These might come in especially handy when you're working on your projects.

Changing level names

Suppose you want to recode the `exerany` variable to use informative level names, instead of 0/1. Remember that this variable indicates whether the respondent exercised in the past month (1) or did not (0). We'll create a new variable called `exerany_cat` and fill it in with `exercised` and `didn't exercise` where we have 1s and 0s, respectively, for the original variable.

```
cdc$exerany_cat = NA # create new variable, and fill it in with NAs first
cdc$exerany_cat[cdc$exerany == 1] = "exercised"
cdc$exerany_cat[cdc$exerany == 0] = "didn't exercise"
cdc$exerany_cat = as.factor(cdc$exerany_cat) # save new variable as factor
```

Exercise 6 Create a box plot of BMI vs. this new variable and confirm that the level names have been changed.

Re-leveling data

You might also want to combine certain levels and re-level a variable. Suppose in the `genhlth` variable, you want to combine the levels `fair` and `poor` to indicate `bad health`, and `good`, `very good`, and `excellent` to indicate `better health`.

```
cdc$genhlth_twolevs = NA # create new variable, and fill it in with NAs first
cdc$genhlth_twolevs[cdc$genhlth %in% c("fair", "poor")] = "bad"
cdc$genhlth_twolevs[cdc$genhlth %in% c("good", "very good", "excellent")] = "better"
cdc$genhlth_twolevs = as.factor(cdc$genhlth_twolevs) # save new variable as factor
```

Exercise 7 Make a plot of this new variable vs. another variable and confirm that the change has been implemented.

We can also re-level numerical variables to be categorical. Suppose we want to create a categorical variable for height, `height_cat`, that identifies people's heights to be below median and at or above median.

```
median_height = median(cdc$height)
cdc$height_cat = NA # create new variable, and fill it in with NAs first
cdc$height_cat[cdc$height < median_height] = "below median"
cdc$height_cat[cdc$height >= median_height] = "at or above median"
cdc$height_cat = as.factor(cdc$height_cat) # save new variable as factor
```

Let's make a table of this new variable to view its distribution:

```
table(cdc$height_cat)
```

The output lists the `at or above median` level first, and `below median` level second. This is because by default R lists levels of a categorical variable in alphabetical order. To change the order, we need to specify the order we would like the levels to be displayed:

```
cdc$height_cat = factor(cdc$height_cat, levels = c("below median", "at or above median"))
```

Exercise 8 Confirm that the order of the levels has been changed by making another table of the `height_cat` variable.

Additional Exercises

Exercise 9 Use one categorical and one numerical variable, "smoke100" and "wt Desire", summarize them and comment on any apparent relationship(s) you see.

Exercise 10 Make the mosaic plot about smoking habits and age. And what can be learned about smoking habits and ages from the mosaic plot? What patterns have you noticed?

Exercise 11 In Exercise 10, the age range is too broad, which detracts from the plot's clarity, and certain age groups have insufficient samples. Please re-create the plot using the code from the previous post to extract an age group you believe is appropriate.

Submission

When you are done, regarding the submission of the lab report, please note the following:

1. The report needs to include Exercise1-13;
2. Please enter your student IDs and names of your group at the top of the report;
3. Please submit the lab report within 24 hours after class;
4. The submitted code must be able to run successfully at one time;
5. Please complete the lab report in HTML and RMD format, package the file into a .zip compressed file and upload it to the Blackboard learning (BB) system, and for each group only one of you send the lab report;
6. Please fill in the name of the report and file in accordance with "student ID 1-name 1-student ID 2-name 2 - lab number - course name". For example, 2021XXXXXXXX-Jack-2021XXXXXXXX -Albert-lab1-Data Analysis and Modeling.

This text refers to some content from Duke University, for which we are grateful.